

AFRILANG Stdlib

std/geo

Français. Bibliothèque AFRILANG 'std/geo' (niveau simple, catégorie utilities). Elle expose 2 fonction(s) : haversineKm, midLat.

'import "std/geo"' · 'use geo'

- 'haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number' - 'midLat(a number, b number) → number' **English.**

AFRILANG library 'std/geo' (tier simple, category utilities). Exports 2 function(s): haversineKm, midLat.

'import "std/geo"' · 'use geo'

- 'haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number' - 'midLat(a number, b number) → number'

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/geo' (niveau simple, catégorie utilities). Elle expose 2 fonction(s) : haversineKm, midLat.

'import "std/geo"' · 'use geo'

```
- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `midLat(a number, b number) → number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/geo"
use geo
```

Niveau : simple · Catégorie : Utilitaires
Helpers simples et fonctions ponctuelles.

3. Référence API

- haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number•
midLat(a number, b number) → number

4. Exemples d'utilisation

```
import "std/geo"
use geo
```

```
create result = haversineKm(/* args */)
say result
```

```
create result = midLat(/* args */)
say result
```

5. Bonnes pratiques

- Préférez `import "std/geo"` en tête de fichier. •
Lancez `afri-lang check` avant de livrer. •
Combinez avec les modules stdlib de la même catégorie. •
Voir /stdlib/ pour le source live et les démos playground. •
Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module geo

```
function haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) returns
  number
end
```

```
function midLat(a number, b number) returns number  
end  
end
```

English

1. Overview

AFRILANG library 'std/geo' (tier simple, category utilities). Exports 2 function(s): haversineKm, midLat.

```
'import "std/geo"' · 'use geo'
```

```
- `haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number`
- `midLat(a number, b number) → number`
```

2. Installation & import

Add the module to your program:

```
import "std/geo"
use geo
```

Tier: simple · Category: Utilities
Simple helpers and single-purpose functions.

3. API reference

- haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) → number•
midLat(a number, b number) → number

4. Usage examples

```
import "std/geo"
use geo
```

```
create result = haversineKm(/* args */)
say result
```

```
create result = midLat(/* args */)
say result
```

5. Best practices

- Prefer `import "std/geo"` at the top of your file.
- Run `afri-lang check` before shipping.
- Combine with related stdlib modules from the same category.
- See /stdlib/ for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module geo

```
function haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) returns
  number
end
```

```
function midLat(a number, b number) returns number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/geo"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/geo/>

Source (extrait)

```
module geo

function haversineKm(lat1 number, lon1 number, lat2 number, lon2 number) returns
    number
end

function midLat(a number, b number) returns number
end
end
```